

ANSWER KEY

Q1. The subquery used in the WHERE clause is a 'Predicate Subquery'. Its purpose is to filter rows in the main query by testing if there exists another row in the self-joined table (sp) with specific conditions: same part number (pno), different supplier numbers (sno), and non-null quantities.

Q2. This SQL query finds the supplier (a row from the "sp" table) with the highest quantity for each part (defined by pno), ignoring rows with null quantities. It does this by first subquerying all non-null qty values for each part and its other suppliers (differing on sno), then comparing each supplier's quantity to those found in the subquery, selecting only the suppliers with a higher quantity.

Q3. Here is the SQL query to list the suppliers who have a higher quantity for each part than any other supplier, ignoring null quantities:

```
```sql

SELECT pno, MAX(sno) AS supplier_id, sno as highest_competitor

FROM sp

WHERE quantity IS NOT NULL AND quantity > (

SELECT MAX(quantity) FROM sp AS tmp WHERE tmp.pno = sp.pno AND tmp.sno != sp.sno

)

GROUP BY pno;

```
```

This query first selects parts with non-null quantities and a higher quantity than any other supplier for that part, then groups by part number (pno). The subquery finds the maximum quantity for each part from suppliers other than the current one (tmp.sno != sp.sno) to compare against the current supplier's quantity (sp.quantity). The MAX(sno) is included in the result set for each part, so you can find the highest competitor for that part if needed.

Q4. Simple (non-correlated) subquery allows various tests and uses all comparison operators. It does not require retrieving only one column in its select list. Correlated subqueries, on the other hand, must be enclosed in parentheses and can use all comparison operators as well. However, they may retrieve multiple columns if needed.

Q5. This SQL statement is looking for part numbers (pno) where the individual quantity in the 'sp' table for each pno is less than the average quantity of that same part number. In other words, it identifies part numbers with an average quantity greater than any individual quantity associated with them in the 'sp' table, excluding the same sequence number (sno).

Q6. (1) Correlated subquery: The subquery is dependent on the outer query's row, as it references column 'a.sno'. It reevaluates for each row of the outer query to find matching rows in the 'sp b' table where the 'pno' is different.

(2) Predicate subquery (scalar): The subquery returns a single value that is compared to other values in the WHERE clause using the operator ALL. In this case, it compares the qty of the current row in 'a' with all the non-NULL qty values from rows where pno, sno are equal and the pno is different from the current row.

Q7. SQL Query:

```
```sql

SELECT p.pno, s.sno
```

```

FROM parts AS p
JOIN (
SELECT pno, MAX(quantity) as max_quantity
FROM parts
WHERE quantity IS NOT NULL
GROUP BY pno
) AS max_q ON p.pno = max_q.pno AND p.quantity = max_q.max_quantity
JOIN suppliers AS s ON p.sno = s.sno;
...

```

This query uses a subquery to find the parts with maximum quantity (ignoring null quantities), then joins back the original 'parts' and 'suppliers' tables on the filtered part numbers and maximum quantity values. The resulting table lists the part number (pno) and supplier number (sno) of suppliers who have the highest quantity for each part.

Q8. ROW EXISTS subquery tests if a subquery retrieves at least one row.

Q9. 3. Self join is NOT a type of predicate subquery. It's actually a form of join used to rejoin a table with itself, not a subquery type.

Q10. CORRELATED, EXISTS, SUBQUERY

These are keywords that can be used for subqueries which refer to columns in both the outer and inner query. This is known as a correlated subquery.

Q11. IN Subquery:

`value NOT IN (subquery)` - Returns rows where the value is not found within the subquery result set.

Example: `SELECT \* FROM Customers WHERE Country NOT IN (SELECT Country FROM Countries WHERE Population > 10000000);`

EXISTS Subquery:

`EXISTS (subquery)` - Returns true if the subquery returns at least one row.

Example: `SELECT Orders.OrderID, Customers.CustomerName FROM Orders INNER JOIN Customers ON Orders.CustomerID = Customers.CustomerID WHERE EXISTS (SELECT \* FROM OrderDetails WHERE Orders.OrderID = OrderDetails.OrderID);`

Q12. A correlated subquery supports various tests using a full set of comparison operators (like =, !=, <, >, <=, >=). Its general format is as follows:

`SELECT column1, column2, ... FROM table WHERE column\_name [OPERATOR] (SELECT expression FROM another\_table)`

Q13. This query selects all rows from the `sp` table where the quantity (`qty`) is greater than all quantities from the same product number (`pno`) but different sequence number (`sno`), excluding any rows where the quantity is NULL.

Q14. a) Returning all rows that have higher quantity than any other row with the same part number (ignoring null quantities).

Q15. Yes, you're correct. The given SQL query will return 't' (true) if there exists a supplier ('P2') who provides the part with color 'Red', and 'f' (false) otherwise.

```sql

```
SELECT EXISTS (SELECT 1 FROM sp WHERE pno = 'P2' AND color = 'Red') AS result
```

```

Q16. This SQL query retrieves rows from table `sp` where quantity (qty) is greater than all other quantities of the same part (pno), excluding null quantities. It does so by performing a self-join on the `sp` table, comparing each row (a) with others having the same part number but different sequential numbers (sno). Rows where the qty is not NULL are considered for comparison.

Q17. ```SQL

```
SELECT s1.sno
```

```
FROM supply_parts s1
```

```
WHERE EXISTS (
```

```
SELECT 1
```

```
FROM supply_parts s2
```

```
WHERE s2.pno = s1.pno AND s2.quantity > s1.quantity AND s2.quantity IS NOT NULL
```

```
) AND s1.quantity IS NOT NULL;
```

```

Q18. The given SQL query 'SELECT * FROM sp a WHERE EXISTS (SELECT * FROM sp b WHERE a.pno = b.pno AND a.sno <> b.sno AND qty IS NOT NULL)' filters rows from the 'sp' table where there is related data for the same part number (pno), different student number (sno), and non-null quantity in another row. It differs from other subqueries as it only tests whether a subquery retrieves at least one row, rather than returning a value or set of values.

Q19. The SQL query to find suppliers who have provided more than 500 units of part P1 is as follows:

```SQL

```
SELECT * FROM sp WHERE EXISTS (SELECT 1 FROM sp b WHERE a.pno = b.pno AND b.pno = 'P1' AND qty > 500)
```

```

Q20. ```SQL

```
SELECT p.pno, p.descr, p.qty, (SELECT SUM(o.qty_ordered) FROM orders o WHERE o.pno = p.pno) as total_ordered, p.qty as in_stock
```

```
FROM parts p
```

```
WHERE total_ordered > in_stock;
```

```

Q21. Represent partial participation of dependents in a policy with a single, dashed line (in UML notation).

Q22. Total Participation: An entity set fully participates (or totally participates) in a relationship set if every entity in the set is related to at least one instance of the relationship. This means that there are no "unmatched" entities within the entity set.

Partial Participation: An entity set partially participates (or incompletely participates) in a relationship set if not every entity in the set is necessarily related to an instance of the relationship. There may be "optional" entities within the entity set.

Q23. In ER (Entity-Relationship) diagrams, Aggregation and Partial Participation represent different types of relationships between entities, but they have distinct visual representations:

1. Aggregation: This relationship indicates a "whole-part" association where the parent entity contains multiple instances of the child entity, and the existence of the child entities does not affect the existence of the whole entity. It is represented by a diamond shape on the parent entity connecting to the child entity line.

2. Partial Participation: This relationship indicates that an instance of the child entity can participate in multiple parent entities at the same time. The degree (number of occurrences) of the child entity is not fixed for each parent entity, and it's possible for a child entity to have no parent entity. It is represented by crow's foot symbols on the child entity line connected to the parent entity.

Q24. Aggregation refers to a "part-of" relationship where one entity contains multiple instances of another, but does not modify them. Partial Participation, on the other hand, allows for optional or multiple occurrences of a weaker entity within a stronger one, without establishing a specific parent-child relationship.

Q25. A Weak Entity Set in ER (Entity-Relationship) diagrams is an entity set that cannot exist independently and must be associated with another entity set through a mandatory, one-to-one relationship to ensure the existence of its primary key. In other words, each weak entity has a partial key, which is a subset of the primary key of the related strong entity.

Q26. Aggregation in ER (Entity-Relationship) modeling represents a whole-part relationship where deletion of the whole may delete the parts. Weak entities, on the other hand, do not exist independently and have a mandatory identifying relationship with another entity (called a strong entity). In contrast to aggregation, the deletion of the strong entity results in the deletion of the weak entity.

Q27. Sure! Here's an example of a SQL function that takes two arguments and returns their sum:

```
```sql
CREATE FUNCTION sum_two_args(arg1 INT, arg2 INT) RETURNS INT
BEGIN
  DECLARE result INT;
  SET result = arg1 + arg2;
  RETURN result;
END;
```
```

This function accepts two integer arguments (``arg1`` and ``arg2``) and returns their sum as an integer. You can use this function in SQL queries like this:

```
```sql
SELECT sum_two_args(3, 5); -- Returns 8
```
```

Q28. `sno='S1' or (city='Rome' and has_car=True)`

Q29. 'a OR (b AND c)' has a higher precedence than 'a AND b OR c'. The parentheses in the second expression ensure that 'b AND c' is evaluated first, and then 'a OR' operation is applied. In the first expression, 'AND' has higher precedence than 'OR', but there are no parentheses to dictate the order of operations explicitly.

Q30. ``(a OR b) AND c``

Q31. `(a OR b) AND c`

Q32. a) The result of 'a AND b OR c NOT d' is True, since 'a' is True and there's no 'AND' condition with 'b' which is False. However, the overall expression is true only if either 'c' (True) or ('a AND b') (False) is True and NOT 'd' (False), but this part of the condition is always true because we don't have 'NOT' before 'd'.

b) The result of 'e OR (f AND g)' is True, since 'e' is False and there's an 'OR' operator that requires only one true value to make the entire expression true. In this case, 'f AND g' returns True as both 'f' and 'g' are True.

Q33. ```SQL

```
SELECT name FROM s
WHERE city IN ('Rome', 'Paris') AND name LIKE '%a%';
```
```

Q34. Sure! Here is an example SQL query using a CASE clause to filter students' grades based on their exam results:

```
```sql
SELECT
student_id,
CASE
WHEN score >= 90 THEN 'A'
WHEN score BETWEEN 80 AND 89 THEN 'B'
ELSE 'C' AS grade
FROM students
WHERE score IS NOT NULL;
```
```

This query will return the student ID and their corresponding grade based on the given rules. It filters out any null scores to avoid any errors in the CASE expression.

Q35. `SELECT * FROM s WHERE name LIKE '%a%';`

This query selects all records from the 's' table where the 'name' column contains the substring 'a'. The '%' symbol acts as a wildcard character, allowing for flexibility in matching the string.

Q36. Scenario 1 (True): A: True, B: True (Output: True)

Scenario 2 (False): A: False, B: False (Output: False)

Scenario 3 (Unknown): A: True, B: Unknown (Output: True if B is true, otherwise unknown)

Q37. ```SQL

`SELECT * FROM Department_of_IT_SVECW WHERE pno = 'P2' OR qty > 200;`

```

Q38. `SELECT * FROM sp WHERE sno = 'S3' OR quantity < 500;`